External Adapters

Purpose

Shio Runtime does not communicate directly with external technologies.

Every interaction with the outside world passes through an Adapter.

Adapters isolate implementation details from the architectural core.

This separation allows technologies to evolve independently while preserving the stability of the Runtime.

What Is an Adapter?

An Adapter is a translation layer between Shio Runtime and an external system.

It converts Runtime concepts into platform-specific operations and converts platform responses back into Runtime concepts.

Adapters never contain business logic.

They never define character behavior.

They never make decisions.

Their responsibility is translation.

Why Adapters Exist

Technologies change.

Architectures should not.

Without Adapters:

```text id="adp01" Speech Runtime

↓

Groq API

↓
```

Application Logic

```
With Adapters:

```text id="adp02"
Speech Runtime

↓

LLM Adapter

↓

Groq
```

Tomorrow:

```text id="adp03" Speech Runtime

↓

LLM Adapter

↓

OpenAI
```

```
Nothing changes inside the Runtime.

Only the Adapter changes.

---

# Categories of Adapters

Different Runtime systems communicate with different categories of external
systems.

---

## AI Adapters

Responsible for communication with language models.

Examples include:

- Groq
- OpenAI
```
```

```
- local inference servers
- future providers

Speech Runtime communicates with the AI Adapter.

Never with the provider directly.

---

## Speech Adapters

Responsible for speech-related services.

Examples include:

- Python Speech
- local ASR services
- local TTS services
- future speech providers

Speech Runtime depends only on the Speech Adapter.

---

## Platform Adapters

Responsible for external platforms.

Examples include:

- YouTube
- Twitch
- Discord
- OBS
- future streaming platforms

Action Runtime communicates through Platform Adapters.

---

## Storage Adapters

Responsible for persistence.

Examples include:

- SQLite
- PostgreSQL
- cloud storage
- vector databases
```

````
Memory Layer communicates only with Storage Adapters.

---

## System Adapters

Responsible for operating system integration.

Examples include:

- filesystem;
- notifications;
- clipboard;
- desktop APIs;
- process execution.

These capabilities remain isolated from Runtime logic.

---

# Adapter Responsibilities

Every Adapter should:

- translate Runtime requests;
- translate external responses;
- normalize errors;
- hide platform-specific APIs;
- expose a stable Runtime interface.

Adapters should never:

- modify Character State;
- execute Runtime logic;
- define personality;
- make architectural decisions.

---

# Replaceability

Every Adapter should be replaceable.

Replacing one provider should never require Runtime redesign.

Examples:

```text id="adp04"
Groq

↓
````
````

```
OpenAI
```

```text id="adp05" Python Speech

↓

Rust Speech Server
```

```
```text id="adp06"
Electron

↓

Tauri
```

Each replacement affects only the Adapter implementation.

The Runtime remains unchanged.

Platform Independence

Action Runtime executes abstract actions.

Adapters translate those actions into platform-specific operations.

For example:

```text id="adp07" BanViewer

↓

Platform Adapter

↓

YouTube API
```

```
or

```text id="adp08"
BanViewer

↓
```

```
Platform Adapter

↓

Twitch API
```

The Runtime understands **BanViewer**.

The Adapter understands the platform.

Error Translation

External systems expose different error models.

Adapters normalize these differences into Runtime concepts.

For example:

```text id="adp09" HTTP 403

↓

PermissionDenied
```

```
```text id="adp10"
Timeout

↓

ActionTimeout
```

The Runtime should never depend on provider-specific error formats.

Adapter Isolation

Adapters must never communicate directly with each other.

For example:

A Platform Adapter should not call a Storage Adapter.

A Speech Adapter should not call an AI Adapter.
```

All coordination occurs inside the appropriate Runtime.

This preserves architectural clarity and prevents hidden dependencies.

Architectural Guarantees

The Adapter architecture guarantees:

- technology independence;
- provider replaceability;
- platform independence;
- simplified testing;
- isolated failures;
- stable Runtime interfaces.

External technologies may evolve continuously.

The Runtime should not.

Future Evolution

As Shio Runtime grows, new external technologies should integrate by adding new Adapters rather than modifying existing Runtime systems.

Future examples include:

- cloud memory providers;
- additional AI providers;
- streaming platforms;
- home automation systems;
- mobile clients;
- web clients.

Regardless of future technologies, one architectural rule remains unchanged:

Runtime systems communicate with architectural concepts. Adapters communicate with technologies.